

Technical Invention Disclosure

Title: System and Method for Automated Compliance Evidence Collection, Cryptographic Integrity Verification, and Continuous Monitoring Using Layer-Aware Freshness Policies, Deterministic Boundary Allocation with Drift Detection, and Dual-Lane Evidence Adjudication

Applicant: MacTech Solutions LLC

Inventor(s): Patrick Caruso

Date of Disclosure: April 15, 2026

Prepared by: Trust Codex Engineering

1. FIELD OF THE INVENTION

This invention relates to computer-implemented systems for regulatory compliance management, specifically to a system and method for automating the collection, verification, scoring, and continuous monitoring of compliance evidence against the NIST SP 800-171 Rev 2 control framework and the Cybersecurity Maturity Model Certification (CMMC) 2.0 standard.

2. BACKGROUND AND PRIOR ART

2.1 Problem Statement

Organizations subject to CMMC 2.0 certification must demonstrate implementation of 110 security controls derived from NIST SP 800-171. Current governance, risk, and compliance (GRC) platforms (e.g., Vanta, Drata, RegScale) provide evidence collection and control tracking but suffer from several deficiencies:

1. **Evidence integrity is not verifiable.** Existing systems store evidence but provide no mechanism to detect post-collection tampering. An auditor cannot cryptographically verify that the evidence set presented at assessment time is identical to what was collected.
2. **Evidence freshness is binary.** Current platforms treat evidence as “collected” or “not collected” without accounting for time-based staleness. A network boundary scan from

6 months ago and one from yesterday are treated identically, despite the former providing substantially weaker assurance.

3. **Control responsibility allocation is opaque.** When organizations use cloud service providers (e.g., Azure Government), certain controls may be fully inherited, shared, or remain customer responsibility. Existing tools either hard-code these assignments or require manual configuration without auditability.
4. **Evidence assessment is monolithic.** Controls that can be satisfied by either technical evidence (automated scans) or policy documentation (governance artifacts) are not tracked independently, forcing artificial “all-or-nothing” assessments.

2.2 Existing Solutions and Their Limitations

PLATFORM	EVIDENCE COLLECTION	INTEGRITY VERIFICATION	FRESHNESS POLICY	ALLOCATION AUDITABILITY	DUAL-LANE EVIDENCE
Vanta	Yes	No	No (binary)	Manual, no hash	No
Drata	Yes	No	No (binary)	Manual, no hash	No
RegScale	Yes	No	Limited	Partial	No
Trust Codex (this invention)	Yes	SHA-256 chain	Layer-aware	Deterministic hash	Yes

3. SUMMARY OF THE INVENTION

The invention comprises four interlocking subsystems that collectively provide a cryptographically verifiable, continuously monitored compliance posture:

Subsystem A — Cryptographic Evidence Attestation Chain: A method for creating tamper-evident compliance evidence bundles using deterministic SHA-256 hashing of sorted evidence fingerprints, coverage summaries, and boundary allocation inputs, enabling post-collection integrity verification.

Subsystem B — Deterministic Boundary Allocation Engine with Drift Detection: A method for automatically allocating regulatory controls to responsibility tiers (Inherited/Shared/Customer/Not Applicable) using a strict precedence hierarchy with gate-dependent service coverage evaluation, producing a deterministic allocation hash for audit stability and drift detection.

Subsystem C — Layer-Aware Evidence Freshness Policy Engine: A method for computing real-time evidence staleness using technology-layer-specific freshness windows (30-180 days), enabling continuous monitoring (COMMON) without batch processing.

Subsystem D — Dual-Lane Evidence Adjudication with Confidence Scoring: A method for independently tracking technical and policy evidence satisfaction for hybrid controls, computing per-control evidence confidence as the minimum of constituent register scores.

4. DETAILED DESCRIPTION

4.1 Subsystem A — Cryptographic Evidence Attestation Chain

4.1.1 Architecture

The system maintains the following data flow:

```

Evidence Collector Run
    |
    v
[Evidence Findings] --mapped--> [Per-Control Results (pass/fail)]
    |
    v
[Enclave Coverage Summary] --bucketed--> [pass_fresh | pass_stale | fail |
no_finding | pass_unknown_layer]
    |
    v
[Coverage Hash] = SHA-256(canonical(source + run_id + fingerprint +
timestamp + totals + per_row(control_id, bucket, layer, freshness_status,
freshness_cutoff)))
    |
    v
[Snapshot Signature] = SHA-256(canonical(boundaryId + allocationHash +
registryVersion + providerProfileId + catalogId +
sorted(evidenceRunFingerprints) + coverageHash))
  
```

4.1.2 Canonical Serialization

All hash inputs are serialized using a deterministic canonical form:

1. Object keys are recursively sorted alphabetically.
2. Arrays are serialized in their natural order (evidence run fingerprints are pre-sorted).
3. Null values are preserved as JSON `null`.
4. The resulting string is UTF-8 encoded and hashed with SHA-256.

This ensures that logically identical inputs always produce identical hashes, regardless of property insertion order in the source language (JavaScript/TypeScript).

Reference Implementation: `computeSnapshotSignature.ts` (lines 34-48), `coverageHash.ts` (lines 8-19)

4.1.3 Tamper Detection

The `verifySnapshotCoverage` function implements post-hoc verification:

1. Retrieve the stored snapshot (including its `coverageHash` and `coverageEvidenceRunId`).
2. Locate the original evidence run by ID or fingerprint.
3. Recompute the coverage summary from the raw evidence findings.
4. Recompute the coverage hash from the fresh summary.
5. Compare the recomputed hash against the stored hash.
6. Return `verified` if hashes match, `hash_mismatch` if they differ (indicating tampering or data corruption).

Reference Implementation: `verifySnapshotCoverage.ts` (lines 89-128)

4.1.4 Coverage Bucketing

Each of the 73 enclave-applicable controls is classified into exactly one of five mutually exclusive buckets:

BUCKET	CONDITION	FRESHNESS STATUS
<code>pass_fresh</code>	Finding exists, pass=true, layer known, within freshness window	<code>fresh</code>
<code>pass_stale</code>	Finding exists, pass=true, layer known, beyond freshness window	<code>stale</code>
<code>pass_unknown_layer</code>	Finding exists, pass=true, layer not mapped	<code>unknown</code>
<code>fail</code>	Finding exists, pass=false	<code>n/a</code>
<code>no_finding</code>	No finding exists for this control	<code>n/a</code>

Reference Implementation: [enclaveCoverage.ts](#) (lines 77-167)

4.2 Subsystem B — Deterministic Boundary Allocation Engine

4.2.1 Allocation Precedence Hierarchy

The engine processes each of 110 NIST controls through a strict three-tier precedence:

```

TIER 1: never_inherited_layers
  └─ If control's ontology layer is in provider's "never inherited" set →
    Customer

TIER 2: always_inherited_layers
  └─ If control's ontology layer is in provider's "always inherited" set
    → Inherited

TIER 3: service_coverage (gate-dependent)
  └─ For each catalog service covering this layer:
    └─ Is the service enabled in boundary input?
    └─ Do all required gates evaluate to "yes"?
    └─ If any active service covers the layer → Shared (never Inherited)

TIER 4: default_allocation
  └─ Fall through to hosting-model-specific default (IaaS/PaaS/SaaS)

```

Critical design constraint: Service coverage NEVER yields “Inherited” status. Even when a cloud provider fully manages a layer (e.g., Azure manages the hypervisor), the customer retains operational responsibility for configuration and monitoring. This prevents false inheritance claims that would leave controls unmonitored.

Reference Implementation: `allocateControls.ts` (lines 44-98)

4.2.2 Gate-Dependent Service Activation

Services contribute to coverage only when: 1. The service is enabled in the boundary input (`services_enabled[key] === true`), AND 2. All required gates for that service evaluate to “yes” in the gate checklist.

Example: Azure Conditional Access covers the “Identity/MFA” layer, but only if the gate `"fedramp_conditional_access_configured"` is answered “yes”. Without the gate, the service is enabled but not active — the control falls through to default allocation.

Reference Implementation: `evaluateGates.ts` (lines 12-32)

4.2.3 Coverage Strength Differentiation

Service coverage includes a `strength` qualifier: - `"platform"`: The provider manages this at the platform level (e.g., physical security). Platform-strength coverage prevents downgrade to Customer even if other gates fail. - `"configuration"`: The provider offers the capability but the customer must configure it (e.g., enabling MFA policies).

Reference Implementation: `allocateControls.ts` (lines 84-87)

4.2.4 Deterministic Allocation Hash

After allocation, all inputs are canonicalized (recursive key sorting) and SHA-256 hashed:

```
AllocationHash = SHA-256(canonical({  
  profile_id,  
  ontology_version,  
  boundaryInput: canonical(boundaryInput),  
  registry_version  
}))
```

This hash serves two purposes: 1. **Audit stability:** The allocation hash is stored on the boundary snapshot. An auditor can verify that the same inputs produce the same allocation. 2. **Drift detection:** When boundary parameters change (e.g., a service is disabled), the new hash differs from the stored hash, triggering a drift alert.

Reference Implementation: [allocationHash.ts](#) (lines 8-17), [drift.ts](#) (lines 5-10)

4.3 Subsystem C — Layer-Aware Evidence Freshness Policy

4.3.1 Freshness Windows by Ontology Layer

Rather than applying a uniform evidence cadence (e.g., “review all evidence monthly”), the system assigns freshness windows based on the control’s ontology layer:

LAYER	FRESHNESS WINDOW	RATIONALE
Identity/MFA	30 days	High-risk attack surface; configuration drift is rapid
Identity/AuthN	30 days	Authentication mechanisms require frequent validation
Identity/Role-Governance	30 days	Privilege creep occurs continuously
Logging/Collection	30 days	Collection failures must be caught quickly
Logging/Monitoring	30 days	Alert rules drift; SIEM configurations change
Logging/Retention	90 days	Retention policies are relatively stable
GuestOS/Patching	30 days	Patch cycles are monthly
GuestOS/Hardening	90 days	Baseline configurations change quarterly
Network/Boundary	90 days	Firewall rules change at moderate cadence
Crypto/Key-Mgmt	180 days	Key rotation cycles are typically semi-annual
Crypto/TLS	180 days	Certificate lifecycles are 90-365 days
Backup/Recovery	90 days	Backup verification is typically quarterly

Reference Implementation: [freshnessPolicy.ts](#) (lines 7-20)

4.3.2 Real-Time Freshness Computation

Freshness is computed at query time, not batch-processed:

```
cutoff = evidence_collected_at + freshness_days_for_layer
status = (now > cutoff) ? "stale" : "fresh"
```

Controls with no layer mapping return `status: "unknown"` with a remediation hint to map the control to an ontology layer.

Reference Implementation: `freshnessPolicy.ts` (lines 33-47)

4.3.3 Integration with Coverage Bucketing

The freshness computation feeds directly into the coverage bucketing (Subsystem A). A finding that passes but is stale is bucketed as `pass_stale` rather than `pass_fresh`, degrading the coverage posture without requiring re-collection.

4.4 Subsystem D — Dual-Lane Evidence Adjudication with Confidence Scoring

4.4.1 Independent Evidence Tracks

The system maintains two independent evidence tracks for each control:

1. **Technical Lane:** Automated scan results (pass/fail/warn/error/na) from evidence collector runs. Stored in `technicalStatus`, `technicalCheckDate`, `technicalEvidenceRunId` fields.
2. **Policy Lane:** Governance document linkages (satisfied/missing/not_required). Stored in `policyStatus`, `policyDocumentIds` fields.

A control's overall status is computed from the combination:

TECHNICAL STATUS	POLICY STATUS	COMBINED CONTROL STATUS
pass	satisfied	pass
pass	missing	partial (policy gap)
fail	satisfied	partial (technical gap)
fail	missing	fail
n/a	satisfied	pass (policy-only control)
pass	not_required	pass (technical-only control)

4.4.2 Hybrid Satisfaction Override

For the ~18 controls requiring both technical and policy evidence, a `hybridSatisfaction` JSONB field allows authorized users to override the combined status:

```
{ "technical": true, "governance": false }
```

This acknowledges that some controls may have compensating controls or alternative evidence that the automated system cannot evaluate.

4.4.3 Register-Based Confidence Scoring

Evidence confidence is computed using operational evidence registers:

1. Each control maps to one or more required registers (e.g., “User_Access_Log”, “Training_Records”).
2. Each register receives a confidence score:
 - **100%** — Finalized entry exists AND is within the register’s cadence window
 - **75%** — Finalized entry exists but is outside the cadence window
 - **50%** — Only draft entries exist
 - **0%** — No entries exist
3. The control’s confidence is the **minimum** of its required registers’ confidence scores (weakest-link model).
4. Technical evidence can boost confidence by up to 10 percentage points (but cannot exceed 100%).

Reference Implementation: `scoring.ts` (lines 51-63, 71-80)

4.4.4 Responsibility Model Integration

When a control’s responsibility model is `"azure_inherited"`, missing operational evidence does not cause a failure status. Instead, the control receives `"na"` status with a note: “Inherited evidence required (provider package).”

This prevents organizations from being penalized for missing evidence that their cloud provider is contractually obligated to supply through their FedRAMP authorization package.

Reference Implementation: `scoring.ts` (lines 131-135)

5. SYSTEM INTEGRATION

The four subsystems operate as an integrated pipeline:



5.1 End-to-End Verification Flow

1. Administrator defines a boundary (cloud provider, services, gate answers).
2. Allocation engine computes control responsibilities and produces an allocation hash.
3. Evidence collector runs scans and produces findings with a run fingerprint.
4. Freshness policy evaluates each finding's staleness based on its ontology layer.
5. Coverage bucketing classifies each control into one of five buckets.
6. Coverage hash is computed from the canonical bucketed summary.
7. Snapshot signature binds: allocation hash + coverage hash + evidence fingerprints.
8. Scoring engine computes per-control confidence from operational registers.
9. At any future point, the verification endpoint can:
 1. Recompute coverage from the original evidence run.
 2. Compare the recomputed hash against the stored snapshot hash.
 3. Report `verified` or `hash_mismatch`.

5.2 Tamper-Resistance Properties

The system provides the following guarantees:

- **Evidence deletion detection:** Removing any finding changes the coverage hash, which breaks the snapshot signature.
 - **Evidence modification detection:** Changing a finding's pass/fail status changes the coverage bucket, which changes the coverage hash.
 - **Allocation drift detection:** Changing any boundary parameter (services, gates, hosting model) changes the allocation hash.
 - **Re-ordering detection:** Evidence run fingerprints are sorted before inclusion in the snapshot signature; any reordering is normalized but any addition/removal is detected.
-

6. CLAIMS OUTLINE

(For patent attorney to formalize)

Independent Claims

Claim 1 (System): A computer-implemented system for automated compliance evidence verification comprising: - a boundary allocation engine that allocates regulatory controls to responsibility tiers using a precedence hierarchy with gate-dependent service evaluation; - an evidence freshness engine that computes per-control evidence staleness using technology-layer-specific freshness windows; - a coverage bucketing engine that classifies controls into mutually exclusive evidence status categories; - a cryptographic attestation engine that produces a tamper-evident snapshot signature by hashing the canonical combination of allocation inputs, coverage summary, and sorted evidence fingerprints; - a verification engine that detects evidence tampering by recomputing coverage from original findings and comparing hashes.

Claim 2 (Method): A computer-implemented method for continuous compliance monitoring comprising the steps of: - receiving boundary parameters including cloud provider profile, service enablement flags, and gate answers; - allocating each regulatory control to a responsibility tier using the precedence hierarchy of Claim 1; - computing a deterministic allocation hash from canonicalized allocation inputs; - receiving evidence findings from collector runs; - computing per-finding freshness status based on the finding's ontology layer and a layer-specific freshness window; - classifying each control into a coverage bucket based on finding existence, pass/fail status, layer mapping, and freshness; - computing a coverage hash from the canonical coverage summary; - producing a snapshot signature binding the allocation hash, coverage hash, and sorted evidence run fingerprints; - storing the snapshot with the signature for subsequent verification.

Claim 3 (Dual-Lane): The system of Claim 1 further comprising a dual-lane evidence adjudication engine that independently tracks technical and policy evidence satisfaction for each control, computing per-control evidence confidence as the minimum confidence score across required operational evidence registers.

Dependent Claims

Claim 4: The system of Claim 1 wherein the allocation engine enforces a constraint that service coverage never yields "Inherited" status, ensuring operational responsibility is never fully delegated.

Claim 5: The system of Claim 1 wherein the freshness engine applies differentiated freshness windows ranging from 30 days for high-risk layers to 180 days for stable layers.

Claim 6: The system of Claim 3 wherein a control with `azure_inherited` responsibility model receives “not applicable” status when operational evidence is missing, rather than “fail” status.

Claim 7: The method of Claim 2 further comprising detecting allocation drift by comparing a newly computed allocation hash against a previously stored allocation hash.

Claim 8: The system of Claim 3 wherein technical evidence passing increases per-control confidence by a bounded increment (up to 10 percentage points) without exceeding 100%.

7. DRAWINGS / FIGURES

(For patent attorney/illustrator to produce — descriptions below)

Figure 1: System architecture block diagram showing the four subsystems and their data flow connections.

Figure 2: Allocation precedence flowchart (never_inherited → always_inherited → service_coverage with gates → default_allocation).

Figure 3: Evidence coverage bucketing decision tree (finding exists? → pass? → layer known? → fresh? → bucket assignment).

Figure 4: Snapshot signature composition diagram showing SHA-256 inputs (allocation hash + coverage hash + sorted fingerprints).

Figure 5: Dual-lane evidence adjudication matrix (technical × policy → combined status).

Figure 6: Register confidence scoring: per-register confidence (100/75/50/0) → minimum across registers → control confidence.

8. REDUCTION TO PRACTICE

The invention has been fully implemented and deployed in production as the Trust Codex platform (<https://cmmc-production.up.railway.app>). The implementation uses:

- **Language:** TypeScript (Node.js 20)
- **Framework:** Next.js 16 (App Router)
- **Database:** PostgreSQL (Railway-hosted)
- **Cryptography:** Node.js `crypto.createHash("sha256")`
- **ORM:** Drizzle ORM

Key source files (all paths relative to project root):

SUBSYSTEM	PRIMARY FILES
A (Attestation Chain)	<code>src/lib/attestation/ computeSnapshotSignature.ts</code> , <code>src/lib/ evidence/verifySnapshotCoverage.ts</code> , <code>src/ lib/evidence/coverageHash.ts</code>
A (Coverage)	<code>src/lib/evidence/enclaveCoverage.ts</code>
B (Allocation)	<code>src/boundary-engine/engine/ allocateControls.ts</code> , <code>src/boundary-engine/ evaluateGates.ts</code>
B (Hash/Drift)	<code>src/boundary-engine/engine/ allocationHash.ts</code> , <code>src/boundary-engine/ engine/drift.ts</code>
C (Freshness)	<code>src/lib/evidence/freshnessPolicy.ts</code>
D (Scoring)	<code>src/lib/evidence-engine/scoring.ts</code>
D (Schema)	<code>src/db/schema.ts</code> (<code>controlRecords</code> , <code>governanceRegisterEntries</code> tables)

First commit containing the invention: available in git history at <https://github.com/WELCOMETOTHE TRIBE/CMMC>

9. INVENTOR DECLARATION

I declare that the technical content described in this disclosure accurately represents the invention as implemented in the Trust Codex codebase. I am the original inventor of the systems described herein.

Signature: _____

Name: Patrick Caruso

Date: _____

10. NOTES FOR PATENT ATTORNEY

1. **Priority date:** File a provisional application (\$320 USPTO) immediately to lock in the priority date. The code is deployed in production and publicly accessible via the Railway URL, which starts the on-sale bar clock.
2. **Prior art search focus:** GRC platforms (Vanta, Drata, RegScale, Hyperproof, Tugboat Logic), OSCAL/FedRAMP automation tools, and general cryptographic evidence chain patents (blockchain-based evidence systems).
3. **Broadening suggestions:**
 - The layer-aware freshness policy is framework-agnostic; broaden claims beyond NIST/CMMC to any regulatory framework with control families.
 - The allocation engine's precedence model applies to any shared-responsibility cloud model (AWS, GCP), not just Azure.
 - The dual-lane adjudication applies to any compliance domain with mixed evidence types (SOC 2, ISO 27001, HIPAA).
4. **Trade secret consideration:** The `control_assessment_logic.v1.json` artifact (which maps controls to required registers and cadence windows) could alternatively be protected as a trade secret rather than disclosed in the patent. Discuss with inventor.
5. **Software patent eligibility (Alice):** The claims should emphasize the technical improvement over prior art (cryptographic tamper detection, deterministic hashing for audit stability) rather than the abstract idea of "compliance management." The specific technical steps (canonical serialization, SHA-256 chaining, layer-specific freshness computation) are concrete technical improvements.